ILLINOIS

Search in course

Search

English

**B**

Hide menu

- **Week 10 Information**
- **Columnar Storage and the modern Data Warehouse**
- **Data Lake**
- **Spark SQL**
- **Other Cloud Analytics**
- **Week 10 Graded Activities**

**Graded Assignment:** Week 10 Quiz

Submitted

**Programming Assignment:** Machine Problem 8 : SparkSQL

. Duration: 4 hours4h

**Programming Assignment:** Machine Problem 9 V3: Analytics - Visualization

. Duration: 6 hours6h

1. CS 498: Cloud Computing Applications
2. Week 10
3. Machine Problem 9 V3: Analytics - Visualization

PreviousNext

**Programming Assignment: Machine Problem 9 V3: Analytics - Visualization**

**Graded · 6/9 points earned**

**Deadline**

Pass this assignment by Apr 20, 11:59 PM CDT

InstructionsMy submissionsDiscussions

**1 Overview**

You have been provided an airlines dataset consisting of 5 CSV files. You need to analyze the data and obtain meaningful insights from it. To do so, you will first perform a simplified version of ETL (Extract, Transform, Load) on these CSV files using AWS Glue. You will then select a subset of this data based on some constraints through AWS Athena using SQL queries. The filtered datasets will then be used inside Tableau.

**2 Requirements**

You will need an AWS account (AWS Athena and Glue are available on the educate accounts) and install Tableau software on your local machine. You have to learn SQL to perform filtering on Athena and get familiar with Tableau. **Please use region -> us-east-1 for your deployment.**

**3 Procedure**

**3.1 Data Upload on AWS S3**

You can download the dataset (flights-dataset.zip) ~ 50MB from below. Once you download this dataset, you have to upload the CSV files to your AWS S3 storage so that the Glue service can access them.

flights-dataset-with-duplicates

ZIP File

If you are not familiar with S3, the official step-by-step tutorial would be helpful.

- S3 Create a bucket tutorial: https://aws.amazon.com/getting-started/hands-on/backup-files-to-amazon-s3/

**3.2 AWS Glue ETL**

To run your ETL jobs, you first need to configure AWS Glue to have the metadata/schema required to read data from S3. You will **create a crawler** to populate the Glue Data Catalog with the metadata for the datastore (S3 Bucket where the flights CSV files are stored acts as a source). You need to attach an IAM role to your crawler with AmazonS3FullAccess and AWSGlueServiceRole policies. This role allows AWS Glue to access your S3 bucket. You also need to specify a database when configuring the crawler. Run your crawler to create a table within the data catalog database specified before. Once the crawler finishes, it will automatically create a table under that database. To ensure the crawler has extracted the correct metadata, manually verify the relevant properties (e.g., data type, attribute names) and schema. You will be using this flights table as input for your first ETL job.

We recommend reading the first two documents to understand the workflow of how AWS Glue populates the data catalog from datastores. The third tutorial talks about how to add a crawler in detail.

- AWS Glue Terminology: https://docs.aws.amazon.com/glue/latest/dg/components-key-concepts.html
- Glue Data Catalog workflow: https://docs.aws.amazon.com/glue/latest/dg/populate-data-catalog.html
- Adding a crawler tutorial: https://docs.aws.amazon.com/glue/latest/ug/tutorial-add-crawler.html

You have already stored the raw data in S3 and created a database with a flights table in the Glue data catalog. Before performing advanced analysis, you must **create two ETL jobs** to transform the data attributes. Glue Studio provides a graphical interface for creating, running, and monitoring your ETL jobs. In the first job, you will utilize AWS Glue Studio to remove some unnecessary columns in the flights table, and filter out some rows that are not required for later analysis. After the first job completes, its output with be stored in another table in the S3 bucket. In the second job, you will write a pyspark script to add a new column to the output table created by the first job. The data for this new column will be

obtained by transforming and combining some of the other existing columns in the table. The output of the second job will be stored in a separate table and will be used later for Athena queries.

### 3.2.1 AWS Glue Job 1

In the first ETL job, you will use AWS Glue Studio to drop columns that are not required. Specifically, you must utilize the flights table created before as a source. Choose ChangeSchema as the transformation method in which the following 17 columns should be kept: **year , month , day , airline , origin_airport, destination_airport , scheduled_departure , departure_time , departure_delay , scheduled_time , elapsed_time , scheduled_arrival , arrival_time , air_time, arrival_delay , diverted , cancelled**, and the remaining columns should be dropped. The input to the ChangeSchema transformation will be the flights data table in the AWS Glue catalog database. (Note: ChangeSchema was previously known as ApplyMapping.)

Once you have dropped the unwanted columns, you can filter out the rows related to the canceled flights, and keep the ones that are not canceled. Choose the Filter transformation method to **keep** the rows that have the attribute **cancelled != 1**. In the Glue Studio, you can connect the output of ChangeSchema transformation as the input of the Filter transformation. This way you can chain the two transformations. The output of the Filter transformation should be stored in a new table in S3 bucket. For this purpose, you **create a table** on Glue Data Catalog (Hint: For the Data Catalog update options, select Create a table in the Data Catalog. Then, choose your database and a new table name. The format should be chosen as JSON, and Compression Type as None. For 'S3 Target Location', you can choose a new S3 folder to store this table). The transformed data which is the output of Glue job 1 will become the input to job 2.

To get started with Glue Studio, the document below is a good reference.

- Glue Studio job example: https://docs.aws.amazon.com/glue/latest/ug/tutorial-create-job.html

### 3.2.2 AWS Glue Job 2

In the second Glue job, you will write a custom PySpark script that will add a new column to the table created as output of the first job. This column will contain the information about the time zone difference between the arrival airport and the departure airport in minutes. For example, if the departure airport is SFO, and arrival airport is ORD, the time zone difference will be 120 minutes. The formula on how to calculate this value by using other columns of the flights table is provided below (here f1 is the first flight).

1

time_zone_difference = (f1.scheduled_arrival / 100)*60 + (f1.scheduled_arrival % 100) - ((f1 .scheduled_departure / 100)*60 + (f1.scheduled_departure % 100) + f1.scheduled_time)%( 24*60)

time_zone_difference = scheduled_arrival - (scheduled_departure + scheduled_time) % 24 hours (i.e. 1440 minuties). Pay attention to the %1440, otherwise you could end up with a wrong number.

**Note:**

Pay attention to the edge case where a flight lands on the next day as this will cause your time zone difference to be off by 24 hours.

Take the floor value after dividing by 100 when doing the calculation.

In AWS Glue console, when creating a job, you can choose the **Spark script editor** option. You can then either upload your script, or use the editor to write the Spark code. The input for job 2 will be the Glue catalog table that was created as the output of job 1. A new **time_zone_difference** column will be added to this data, and the result will be stored as a new Glue catalog table. The output data should be stored in a new location / folder in the S3 bucket. Follow the same hint as in job 1 for storing the results of job 2. This new table you created through AWS Glue job 2 will be used in Athena queries described in later sections.

**Note:** For repeated runs of job 2, you will have to manually or programmatically edit the schema and delete the 'time_zone_difference' column from the Glue catalog table. Otherwise when you run job 2, it will fail with the error that column already exists. Also you can clean any intermediate data stored in the S3 bucket folders as a result of running jobs 1 and 2. This maybe important to get the correct counts in the Athena queries.

The links below refers to a document that provides information on how to write AWS Glue ETL scripts in PySpark.

- AWS Glue ETL scripts in PySpark: https://docs.aws.amazon.com/glue/latest/dg/aws-glue-programming-python.html
- Creating tables and updating schema in Data Catalog: https://docs.aws.amazon.com/glue/latest/dg/update-from-job.html

You can use the following API reference for AWS Glue client.

- Boto3 Glue Client: https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/glue.html

The template PySpark script file below should help you get started with this script.

job2_template

PY File

### 3.2.3 AWS Glue Job 3

There is a third type of Glue job known as “Notebooks” or “interactive session”.

You may find the documentation here: https://docs.aws.amazon.com/glue/latest/dg/interactive-sessions-chapter.html

Using interactive sessions, i.e., Jupyter notebooks provided by Glue, perform the following tasks. You have to load the data (which should be the output of job #2) from the data catalog as an input for this job. If you have trouble reading the updated data from job 2 in job 3, try crawling the S3 folder again in which the output of Job 2 was stored. Stored this crawled data in a new database and table in the data catalog.

**1. Deduplication**

Your goal is to identify duplicate records and analyze the impact of deduplication on data volume and distribution. Report the number of duplicates removed and changes in metrics - mean and standard deviation for the given numerical columns:

departure_delay,

air_time,

arrival_delay

Find mean and std deviation for each of these columns before and after the removal of duplicates. Make sure to round your answers to the nearest whole number both times.

Find the change in mean for each of the three columns and add the change together.

Eg:

Means before removal of duplicates:

Departure_delay mean = 2

Air_time mean = 2

Arrival_delay  mean = 3

After removal of duplicates:

Departure_delay mean = 1

Air_time mean = 4

Arrival_delay  mean = 7

Find the change:

Departure_delay diff = |1-2| = 1

Air_time diff = |4-2| =  2

Arrival_delay diff = |7-3| = 4

Total change for mean = 1+2+4 = 7

Do the same for standard deviation.

Report your answers in the form of a list in the given format:

[Number of duplicates, total change for mean, total change for stddev]

**2. Flight Route Popularity Analysis**

To Determine the most popular flight routes within the dataset.

Use a Glue job to combine the "Origin" and "Destination" columns, creating a new column called "Route" (e.g., 'ORD-JFK', 'LAX-SFO'). Note: No whitespaces in the values stored in Route column.

Group the data by the "Route" column.

Count the number of occurrences of each route, representing flight frequency.

Sort the aggregated route data in descending order by frequency.

Extract the top-10 routes.

Store the list as a csv file eg:

route,count

ORD-JFK,1000

LAX-SFO, 900

.....

You should be able to download the csv file locally. In your submission, provide the local path to this csv file.

**3. Correlation: Finding which variable affects the departure delay the most.**

Calculate the correlation between departure_delay and some these numerical columns:

'scheduled_time', 'elapsed_time', 'diverted', 'departure_time', 'day'.

Report which numerical variable (as a string in lowercase) affects the departure delay the most.

**4. Time-of-Day Bucketing for Delay Analysis**

Create a new column categorizing departure times into broad buckets (e.g., "Morning", "Afternoon", "Night") to investigate variations in flight delays throughout the day using the 'departure_time' column.

Utilize a custom mapping function within pyspark to extract only the hour component from the 'DepartureTime' column.

Implement conditional logic to set up your buckets.

The buckets are as follows:

5 <= hour <= 11: Morning

12 <= hour <= 17: Afternoon

18 <= hour <= 23 or 0 <= hour <= 4: Night

For all other cases, you may label it as Unknown.

Direct this categorized time-of-day output to a new column named 'DepartureTimeBucket'. We will use this column in one of the Athena queries.

After all these tasks are completed, store your final, transformed data in a new S3 bucket.

Crawl this new S3 bucket data into a new table inside a new Glue Data Catalog database.

We will use this final output table in the Data Catalog in all of the Athena queries.

**3.3 SQL Query and Filtering through AWS Athena**

- What is AWS Athena: https://docs.aws.amazon.com/athena/latest/ug/what-is.html

- SQL Reference for Athena: https://docs.aws.amazon.com/athena/latest/ug/ddl-sql-reference.html

In Athena, you should be able to view the database and table created through the Glue, reflecting the total entries of **471167** (with header trimmed). One way to verify the number is to write a simple SQL query with aggregate functions and check the result. Here the table name would be for the AWS Glue job 3 output table.

1

2

```
SELECT COUNT(*)

FROM "<YOUR_DATABASE_NAME>"."<YOUR_TABLE_NAME>";
```

### 3.3.1 SQL Query 1

You must **write a SQL query** for the following conditions:

Find all possible flights that schedule to depart from ORD between 8 AM and 12 PM, [8AM, 12PM), on Christmas (12/25):

1. In your dataset, scheduled_time, elapsed_time, departure/arrival delays are in **MINUTES**, while scheduled_departure, scheduled_arrival, departure/arrival_time are in **HHMM**. You could look at your data to verify it and handle the format.

3. Note that the time range is **[8AM, 12PM)**. That is, 12:00 PM is excluded from the calculation.

You need to output the following three columns for each flight: **Airline, Origin_Airport, Destination_ Airport**.

Note: There should be one row for each flight in the output. Please **save the count of entries (number)** you obtain from the query as you would need to **submit this parameter to the autograder**. We suggest testing the result on autograder before going to the next part. Please refer to the submission section at the bottom. The output CSV file from this query will be used by Tableau for the visualization task.

### 3.3.2 SQL Query 2

You must **write a SQL query** for the following conditions:

Select all possible trips from SFO to JFK with one stop-over airport in between such that:

1. 60 minutes <= stop-over time <= 180 minutes. Here, the stop-over time is the difference between the arrival time of the first flight and the departure time of the second flight. Note that we are considering actual arrival time and departure time (not scheduled arrival time or scheduled departure time). Look at the stop-over time calculation in the next section.
2. The two flights on a given trip should belong to the same airline.
3. Neither of the two flights was canceled.
4. Note that we may ignore many real-world conditions. As a result, simply conform to the aforementioned conditions.

You need to output the following eight columns for each trip: **Airline, Origin_Airport, Stopover_Airport, Destination_ Airport, Origin_Departure_Delay, Stopover_Arrival_Delay, Stopover_Departure_Delay, Destination_Arrival_Delay**.

Note: There should be one row for each trip in the output. Please **save the count of entries (number)** you obtain from the query as you would need to **submit this parameter to the autograder**. We suggest testing the result on autograder before going to the next part. Please refer to the submission section at the bottom. The output CSV file from this query will be used by Tableau for the visualization task.

**Details about stop-over time calculation**

Stop-over time pseudo-code:

1

2

3

4

5

6

-- f1 is flight1; f2 is flight2

-

- stop_over_time = f2.day*24*60 + (f2.scheduled_departure / 100)*60 + (f2.scheduled_departure % 100) + f2.departure_delay - (f1.day*24*60 + (f1.scheduled_departure / 100)*60 + (f1.scheduled_departure % 100) + f1.departure_delay + f1.elapsed_time + f1.time_zone_difference)

SELECT ...

FROM ...

WHERE 60 <= stop_over_time AND stop_over_time <= 180 AND ...

**Note**:

- Make sure you **read and understand** the below explanation before using the code.
- Pay attention to the timing format. It is helpful to transform **Day** and **HHMM** attributes into **MINUTES**. For example, Day 16 could be 16*24*60 minutes, while 1820 (i.e. 18:20 or 06:20 PM) could be (1820/100)*60 + (1820%100) = 18*60 + 20 = 1100 minutes

1. The DAY attribute reflects the date of the scheduled departure. The actual departure/arrival date could be different.

2. You could use the scheduled departure time, the departure delay, and the elapsed time to get the actual arrival time.

3. There could be a time zone difference between the origin and the destination. All times are local.

- arrival_time (local) = scheduled_departure+ departure_delay + elapsed_time +time_zone_diff
- scheduled_arrival (local) = scheduled_departure + scheduled_time +time_zone_diff
- departure_time (local) = scheduled_departure + departure_delay
- stop-over time = departure_time (local) - arrival_time (local)

You could use these expressions and transform all of the formats of timings into minutes to deal with dates changes.

4. Some of the formats of timings in the dataset are in integer format as **HHMM** (i.e., 1730 means 5:30 PM), and some are in **MINUTES**. If a flight is scheduled at 23:50 (shown as 2350) and delayed for 20 minutes, the departure time will be 00:10 (shown as 10), then the DAY column value does not reflect the actual departure date.

5. Example:

Note: We use **HH:MM** in the following example for clarity.

Flight 1:

day scheduled_departure departure_time scheduled_time elapsed_time schedule d_arrival arrival_time

1 23:50 00:10 04:10 03:40 06:00 0 5:50

The departure_time, scheduled_arrival, and arrival_time are all on date = 2nd.

time_zone_difference = 2 hours (i.e. 120 minutes). This value you have already calculated and stored in a column as part of Glue ETL job 2 above. In the query you will just read this value from the time_zone_difference column and shown in the pseudo-code above.

arrival_time (local)= Dec. 1st (day) + 23:50 (scheduled_departure) + 00:20 (departure_delay) + 03:40 (elapsed_time) + 02:00 (time_zone_diff) = Dec. 1st + 29:50 = Dec. 2nd + 05:50

scheduled_arrival (local) = Dec. 1st (day) + 23:50 (scheduled_departure) + 04:10 (scheduled_time) + 02:00 (time_zone_diff) = Dec. 1st + 30:00 = Dec.2nd + 06:00

Flight 2:

day scheduled_departure departure_delay departure_time

2 07:50 00:10 08:00

departure_time (local) = Dec. 2nd (day) + 07:50 (scheduled_departure)+ 00:10(departure_delay) = Dec. 2nd + 08:00

stop-over time = flight2_depature_time - flight1_arrival_time = Dec. 2nd + 08:00 - (Dec. 1st + 29:50) = Dec. 2nd + 08:00 - (Dec. 2nd + 05:50) = 02:10 (i.e. 130 minutes)

**3.3.3 SQL Query 3**

Now in Athena using the column named 'DepartureTimeBucket', investigate whether specific times of day experience higher average ARRIVAL delays and flight volumes, uncovering potentially congested hours at airports. You have to find which time of day - "Morning", "Afternoon", "Night" has the highest average arrival delay (rounded to the nearest whole number). Your deliverable should look something like this:

{"Morning": 10}

**3.4 Tableau Setup**

You are eligible for a free Tableau Desktop license through Tableau for StudentsOpen as a student. To download and install Tableau, you must first fill out this form, providing your @illinois.edu university email address.

Next, you will receive an email on your email account with download links along with the product key. You will be using the product key during installation.

Tableau Desktop can only be installed on macOS (version >=10.13) or Windows. You may have to use virtual machines if you are using other OSes.

If you would like to get more info on using Tableau, please check out the Tableau Tutorial. You can also refer to the first 30 minutes of this video.

### 3.5 Tableau Visualization

#### 3.5.1 Task 1

With the output CSV files from **SQL query 1**, you need to complete one visualization task.

In the Tableau window, click on the File menu and then New. Next, you need to connect to the CSV file. Once connected, you create one worksheet for the task below and save the results as mentioned.

1. For the flight departing from ORD on Christmas, create a symbol map to show the count of the airlines that the destination airport accommodates. Select Tableau geographic role with the airport format on "Destination Airport" to auto-generate the latitude and longitude.

Save the data corresponding to this visualization as follows: Right-click on the visualization, then click "View Data" (4 columns: destination airports, count of airlines, latitudes, and longitudes), ensure that "Summary Data" is selected at the bottom, and finally click on "Export All" to save the data into a file named “mp9-viz0.tsv” or “mp9-viz0.csv” depending on whether your file is tab or comma-separated.

The output figure might look like the below:

#### 3.5.2 Task 2

You must complete two visualization tasks with SQL query 2.

In the Tableau window, click on the File menu and then New. Next, you need to connect to the CSV file. Once connected, you create two worksheets for each of the tasks below and save the results as mentioned.

1. Create a bar chart to show the count of flights between SFO and different stop-over airports.
2. Create a scatter plot with Stop-over Departure Delay at the intermediate airport as x-axis and Destination Arrival Delay at JFK as Y-axis. Now create two filters: one for Stop-over Departure Delay and another for Destination Arrival Delay. Set the range of Departure Delay filter from 200 to 260 and Arrival Delay filter from 110 to 220.

Save the data corresponding to this visualization as follows: Right-click on the visualization, then click "View Data", ensure that "Summary Data" is selected at the bottom, and finally click on "Export All" to save the data into a file named “mp9-viz1.tsv” or “mp9-viz1.csv” and “mp9-viz2.tsv” or “mp9-viz2.csv” depending on whether your file is tab or comma-separated.

The output figures might look like the below:

1. x-axis is the count of flights, and the y-axis is stop-over airports

2. x-axis is Stop-over Departure Delay, and the y-axis is Destination Arrival Delay.

**4 Submission**

You need to download the test.py file attached below to submit your work. You must modify the required parameters within test.py that take in your email address, Coursera secret token, output of 3.2.3 AWS Glue Job 3 part 1, 2, 3 and the number of filtered entries from 3.3 along with the three file paths corresponding to the CSV files from 3.5. Please also double-check the viz0 column indices, viz1StopOverColumn and viz2ArrivalDelayColumn.

If you only want to test tasks in 3.2.3 AWS Glue Job 3 except the top N routes, please replace topndata with empty dict in submission script.

```
1

2

def main():

  topndata = {} # route_analysis(etl3_job2, route, count)
```

Similarly, if you only want to test the number of filtered entries in 3.3, please replace viz0Data, viz1Data, and viz2Data with empty dicts. Don't forget to revert them back while testing the visualization exercises in 3.5.

1

2

3

4

5

6

```
def main():
   topndata = route_analysis(etl3_job2, route, count)
   viz0Data = {} #readViz0(viz0CsvPath, viz0DestAirportColumn, viz0CountAirlineColumn, viz0LatitudeColumn, viz0LongitudeColumn)
   viz1Data = {} #readViz12(viz1CsvPath, viz1StopOverColumn)
   viz2Data = {} #readViz12(viz2CsvPath, viz2ArrivalDelayColumn)
   ...
```

The delimiter used in the output file from tableau can be **','** instead of '**\t**'. However, test.py uses '\t' as the delimiter by default. If your output is comma-separated, please update the delimiter inside test.py at **line 55 and line 70** as follows:

Change:

reader = csv.reader((line.replace('\0','') for line in csvfile), **delimiter='\t')**

to

reader = csv.reader((line.replace('\0','') for line in csvfile), **delimiter=',')**

Execute test.py, which will update your grade or provide you with an error message.

mp9_v3_test

PY File

**5 FAQs and Resources**

**FAQs**

1. When passing strings to a SQL statement in Athena, they need to be in single quotes so that they don't get mistaken for a column label which is in double quotes.
2. ChangeSchema was previously called ApplyMapping.
3. Check mark the update schema radio button in last step of job 1.
4. To debug and see where the error occurred, go to the “continuous logs” section of the job.
5. Deleting intermediate results (only) in the s3 bucket is important.
6. If records are missing values in some relevant fields, you can ignore them, you do not have to explicitly handle such cases.
7. Take the floor value after dividing by 100 when doing the calculation in SQL queries. It is already illustrated in the example given in the instructions.

**Resources**

1. Spark scripts how to: https://docs.aws.amazon.com/glue/latest/dg/aws-glue-programming-intro-tutorial.html
2. Tableau video tutorials: https://www.tableau.com/learn/training

**How to submit**

Copy the token below and run the submission script included in the assignment download. When prompted, use your email address **fridkis2@illinois.edu**.

Your token expired

Generate new token

Your submission token is unique to you and should not be shared with anyone. You may submit as many times as you like.

Like

Dislike

Report an issue